

```
from google.colab import files
uploaded = files.upload()
```



Choose Files archive.zip

- **archive.zip**(application/x-zip-compressed) - 208660586 bytes, last modified: 6/5/2024 - 100% done
- Saving archive.zip to archive.zip

```
import zipfile
import os

# Change 'your_zip_file.zip' to the name of your uploaded zip file
zip_file = 'archive.zip'
destination_dir = 'images'

with zipfile.ZipFile(zip_file, 'r') as zip_ref:
    zip_ref.extractall(destination_dir)

# Verify the contents of the directory
for root, dirs, files in os.walk(destination_dir):
    print(f'Directory: {root}')
    for file in files:
        print(f'  File: {file}')
```



```
File: image_18.1.jpg
File: img183.jpg
File: 25.6.jpg
```

```
from google.colab import drive
drive.mount('/content/drive')
```

 Mounted at /content/drive

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.applications import VGG16
from tensorflow.keras.callbacks import ModelCheckpoint, ReduceLROnPlateau
```

```
# Directory for the dataset
data_dir = '/content/images/PSORIASIS AND NORMAL SKIN'
```

```
# Data generators
datagen = ImageDataGenerator(rescale=1./255, rotation_range=20, width_shift_range=0.2,
                             height_shift_range=0.2, shear_range=0.2, zoom_range=0.2,
                             horizontal_flip=True, fill_mode='nearest', validation_split=0.3)
```

```
train_generator = datagen.flow_from_directory(data_dir, target_size=(128, 128),
                                              batch_size=32, class_mode='binary', subset='training')
validation_generator = datagen.flow_from_directory(data_dir, target_size=(128, 128),
                                                  batch_size=32, class_mode='binary', subset='validation')
```

```
# Splitting validation data further into validation and test sets
validation_datagen = ImageDataGenerator(rescale=1./255, validation_split=0.5)
validation_generator = validation_datagen.flow_from_directory(data_dir, target_size=(128, 128),
                                                            batch_size=32, class_mode='binary', subset='validation')
test_generator = validation_datagen.flow_from_directory(data_dir, target_size=(128, 128),
                                                       batch_size=32, class_mode='binary', subset='training')
```

 Found 1965 images belonging to 2 classes.
Found 841 images belonging to 2 classes.
Found 1403 images belonging to 2 classes.
Found 1403 images belonging to 2 classes.

```
# Define the model
base_model = VGG16(include_top=False, input_shape=(128, 128, 3))
base_model.trainable = False
```

```
model = Sequential([
    base_model,
    Flatten(),
    Dense(256, activation='relu'),
    Dropout(0.5),
    Dense(1, activation='sigmoid')
])
```

```
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
```

```
# Modify Callbacks
model_checkpoint = ModelCheckpoint('skin_disease.h5', save_best_only=True, monitor='val_loss', verbose=1)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.2, patience=5, verbose=1, min_lr=0.0001)
```

```
# Train the model
history = model.fit(train_generator, validation_data=validation_generator, epochs=30,
                   callbacks=[model_checkpoint, reduce_lr])
```

```
# Evaluate the model on the test set
model.load_weights('skin_disease.h5')
test_loss, test_acc = model.evaluate(test_generator)
print(f'Test accuracy: {test_acc}')
```



```

Epoch 19: val_loss did not improve from 0.05477
62/62 [=====] - 19s 304ms/step - loss: 0.0706 - accuracy: 0.9735 - val_loss: 0.1446 - val_accuracy: 0.9480 -
Epoch 20/30
62/62 [=====] - ETA: 0s - loss: 0.0533 - accuracy: 0.9817
Epoch 20: val_loss did not improve from 0.05477
62/62 [=====] - 19s 309ms/step - loss: 0.0533 - accuracy: 0.9817 - val_loss: 0.0640 - val_accuracy: 0.9772 -
Epoch 21/30
62/62 [=====] - ETA: 0s - loss: 0.0483 - accuracy: 0.9852
Epoch 21: val_loss did not improve from 0.05477
62/62 [=====] - 21s 340ms/step - loss: 0.0483 - accuracy: 0.9852 - val_loss: 0.1205 - val_accuracy: 0.9551 -
Epoch 22/30
62/62 [=====] - ETA: 0s - loss: 0.0635 - accuracy: 0.9791
Epoch 22: val_loss did not improve from 0.05477

Epoch 22: ReduceLROnPlateau reducing learning rate to 0.0002000000949949026.
62/62 [=====] - 19s 311ms/step - loss: 0.0635 - accuracy: 0.9791 - val_loss: 0.0857 - val_accuracy: 0.9694 -
Epoch 23/30
62/62 [=====] - ETA: 0s - loss: 0.0461 - accuracy: 0.9832
Epoch 23: val_loss did not improve from 0.05477
62/62 [=====] - 19s 312ms/step - loss: 0.0461 - accuracy: 0.9832 - val_loss: 0.0990 - val_accuracy: 0.9629 -
Epoch 24/30
62/62 [=====] - ETA: 0s - loss: 0.0381 - accuracy: 0.9863
Epoch 24: val_loss did not improve from 0.05477
62/62 [=====] - 19s 314ms/step - loss: 0.0381 - accuracy: 0.9863 - val_loss: 0.0932 - val_accuracy: 0.9651 -
Epoch 25/30
62/62 [=====] - ETA: 0s - loss: 0.0313 - accuracy: 0.9873
Epoch 25: val_loss did not improve from 0.05477
62/62 [=====] - 20s 322ms/step - loss: 0.0313 - accuracy: 0.9873 - val_loss: 0.1002 - val_accuracy: 0.9651 -
Epoch 26/30
62/62 [=====] - ETA: 0s - loss: 0.0321 - accuracy: 0.9919
Epoch 26: val_loss did not improve from 0.05477
62/62 [=====] - 21s 343ms/step - loss: 0.0321 - accuracy: 0.9919 - val_loss: 0.0720 - val_accuracy: 0.9743 -
Epoch 27/30
62/62 [=====] - ETA: 0s - loss: 0.0378 - accuracy: 0.9878
Epoch 27: val_loss did not improve from 0.05477

Epoch 27: ReduceLROnPlateau reducing learning rate to 0.0001.
62/62 [=====] - 19s 305ms/step - loss: 0.0378 - accuracy: 0.9878 - val_loss: 0.0616 - val_accuracy: 0.9779 -
Epoch 28/30
62/62 [=====] - ETA: 0s - loss: 0.0329 - accuracy: 0.9878
Epoch 28: val_loss did not improve from 0.05477
62/62 [=====] - 20s 323ms/step - loss: 0.0329 - accuracy: 0.9878 - val_loss: 0.0890 - val_accuracy: 0.9658 -
Epoch 29/30
62/62 [=====] - ETA: 0s - loss: 0.0299 - accuracy: 0.9898
Epoch 29: val_loss did not improve from 0.05477
62/62 [=====] - 21s 337ms/step - loss: 0.0299 - accuracy: 0.9898 - val_loss: 0.0933 - val_accuracy: 0.9658 -
Epoch 30/30
62/62 [=====] - ETA: 0s - loss: 0.0280 - accuracy: 0.9908
Epoch 30: val_loss did not improve from 0.05477
62/62 [=====] - 20s 322ms/step - loss: 0.0280 - accuracy: 0.9908 - val_loss: 0.0764 - val_accuracy: 0.9743 -
44/44 [=====] - 5s 104ms/step - loss: 0.0188 - accuracy: 0.9936
Test accuracy: 0.9935851693153381

```

```

# Evaluate the model on the validation data
validation_loss, validation_accuracy = model.evaluate(validation_generator)
print(f'Validation accuracy: {validation_accuracy}')

```

```

➡ 27/27 [=====] - 2s 80ms/step - loss: 0.0739 - accuracy: 0.9715
Validation accuracy: 0.9714625477790833

```

```

from sklearn.metrics import confusion_matrix, classification_report
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import load_model

```

```

# Load your trained model
model = load_model('/content/skin_disease.h5')

```

```

# Data generators
datagen = ImageDataGenerator(rescale=1./255, validation_split=0.3)
validation_generator = datagen.flow_from_directory(data_dir, target_size=(128, 128),
                                                    batch_size=32, class_mode='binary', subset='validation')

```

```

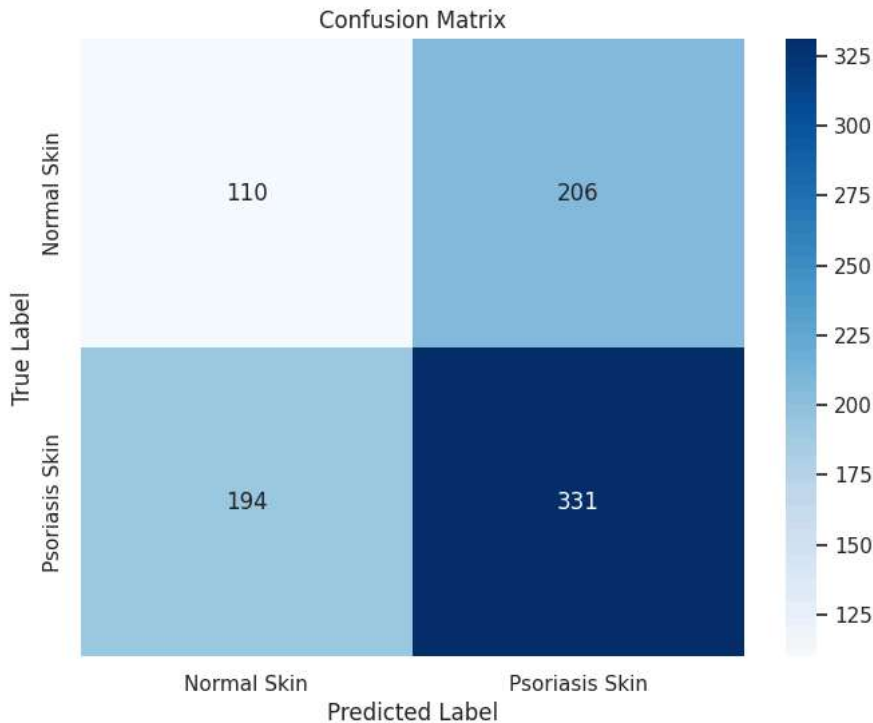
# Generate predictions on validation data
validation_generator.reset()
Y_pred = model.predict(validation_generator)
y_pred = (Y_pred > 0.5).astype("int32")

```

```
# Compute confusion matrix
cm = confusion_matrix(validation_generator.classes, y_pred)
cm_labels = ['Normal Skin', 'Psoriasis Skin'] # Adjust this according to your class labels

# Plot confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt="d", cmap='Blues', xticklabels=cm_labels, yticklabels=cm_labels)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Confusion Matrix')
plt.show()
```

Found 841 images belonging to 2 classes.
27/27 [=====] - 2s 78ms/step



```
# Assuming you have the `history` object from training
def plot_training_history(history):
    sns.set(style="whitegrid")

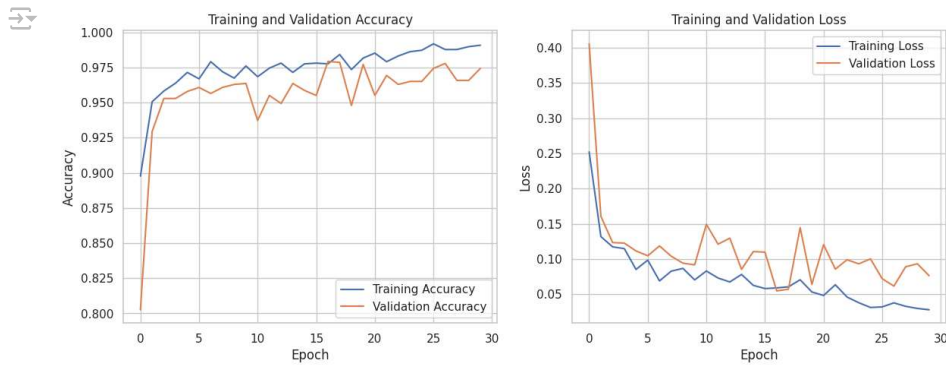
    plt.figure(figsize=(14, 5))

    # Plot training & validation accuracy values
    plt.subplot(1, 2, 1)
    plt.plot(history.history['accuracy'], label='Training Accuracy')
    plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
    plt.xlabel('Epoch')
    plt.ylabel('Accuracy')
    plt.legend(loc='lower right')
    plt.title('Training and Validation Accuracy')

    # Plot training & validation loss values
    plt.subplot(1, 2, 2)
    plt.plot(history.history['loss'], label='Training Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')
    plt.xlabel('Epoch')
    plt.ylabel('Loss')
    plt.legend(loc='upper right')
    plt.title('Training and Validation Loss')

    plt.show()

# Assuming you have already trained the model and have the `history` object
plot_training_history(history)
```



```

from google.colab import files
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing import image
from tensorflow.keras.models import load_model
import numpy as np

# Load your trained model
model = load_model('/content/skin_disease.h5')

# Function to load and preprocess the image
def load_and_preprocess_image(img_path, target_size=(128, 128)):
    img = image.load_img(img_path, target_size=target_size)
    img_array = image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0)
    img_array /= 255.0 # Assuming you normalized your images during training
    return img_array

# Function to predict the class of the image
def predict_image_class(model, img_array):
    prediction = model.predict(img_array)
    predicted_class = (prediction > 0.5).astype("int32")
    return predicted_class

# Function to print the prediction result
def print_prediction_result(predicted_class):
    class_labels = ['Normal Skin', 'Psoriasis Skin'] # Adjust this according to your class labels
    print(f"The uploaded image is classified as: {class_labels[predicted_class[0][0]]}")

# Use Colab's file upload widget
uploaded = files.upload()

for filename in uploaded.keys():
    img_path = filename
    img_array = load_and_preprocess_image(img_path, target_size=(128, 128)) # Adjust target_size
    predicted_class = predict_image_class(model, img_array)
    print_prediction_result(predicted_class)

# Display the uploaded image
img = image.load_img(img_path)
plt.imshow(img)
plt.axis('off')
plt.show()

```



Choose Files seborrheic-...atit-86.jpg

• **seborrheic-dermatitis-86.jpg**(image/jpeg) - 75258 bytes, last modified: 6/13/2024 - 100% done

Saving seborrheic-dermatitis-86.jpg to seborrheic-dermatitis-86.jpg

1/1 [=====] - 0s 156ms/step

The uploaded image is classified as: Psoriasis Skin



```
from google.colab import files
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing import image
from tensorflow.keras.models import load_model
import numpy as np

# Load your trained model
model = load_model('/content/skin_disease.h5')

# Function to load and preprocess the image
def load_and_preprocess_image(img_path, target_size=(128, 128)):
    img = image.load_img(img_path, target_size=target_size)
    img_array = image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0)
    img_array /= 255.0 # Assuming you normalized your images during training
    return img_array

# Function to predict the class of the image
def predict_image_class(model, img_array):
    prediction = model.predict(img_array)
    predicted_class = (prediction > 0.5).astype("int32")
    return predicted_class

# Function to print the prediction result
def print_prediction_result(predicted_class):
    class_labels = ['Normal Skin', 'Psoriasis Skin'] # Adjust this according to your class labels
    print(f"The uploaded image is classified as: {class_labels[predicted_class[0][0]]}")

# Use Colab's file upload widget
uploaded = files.upload()

for filename in uploaded.keys():
    img_path = filename
    img_array = load_and_preprocess_image(img_path, target_size=(128, 128)) # Adjust target_size
    predicted_class = predict_image_class(model, img_array)
    print_prediction_result(predicted_class)

# Display the uploaded image
img = image.load_img(img_path)
plt.imshow(img)
plt.axis('off')
plt.show()
```



Choose Files psoriasis-p...oles-143.jpg

• **psoriasis-palms-soles-143.jpg**(image/jpeg) - 60369 bytes, last modified: 6/13/2024 - 100% done

Saving psoriasis-palms-soles-143.jpg to psoriasis-palms-soles-143.jpg

1/1 [=====] - 0s 356ms/step

The uploaded image is classified as: Psoriasis Skin



```
from google.colab import files
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing import image
from tensorflow.keras.models import load_model
import numpy as np

# Load your trained model
model = load_model('/content/skin_disease.h5')

# Function to load and preprocess the image
def load_and_preprocess_image(img_path, target_size=(128, 128)):
    img = image.load_img(img_path, target_size=target_size)
    img_array = image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0)
    img_array /= 255.0 # Assuming you normalized your images during training
    return img_array

# Function to predict the class of the image
def predict_image_class(model, img_array):
    prediction = model.predict(img_array)
    predicted_class = (prediction > 0.5).astype("int32")
    return predicted_class

# Function to print the prediction result
def print_prediction_result(predicted_class):
    class_labels = ['Normal Skin', 'Psoriasis Skin'] # Adjust this according to your class labels
    print(f"The uploaded image is classified as: {class_labels[predicted_class[0][0]]}")

# Use Colab's file upload widget
uploaded = files.upload()

for filename in uploaded.keys():
    img_path = filename
    img_array = load_and_preprocess_image(img_path, target_size=(128, 128)) # Adjust target_size
    predicted_class = predict_image_class(model, img_array)
    print_prediction_result(predicted_class)

# Display the uploaded image
img = image.load_img(img_path)
plt.imshow(img)
plt.axis('off')
plt.show()
```

Choose Files 123.jpg

• 123.jpg(image/jpeg) - 330196 bytes, last modified: 7/5/2024 - 100% done

Saving 123.jpg to 123.jpg

WARNING:tensorflow:5 out of the last 58 calls to <function Model.make_predict_function.<

1/1 [=====] - 0s 145ms/step

The uploaded image is classified as: Psoriasis Skin



```
from google.colab import files
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing import image
from tensorflow.keras.models import load_model
import numpy as np

# Load your trained model
model = load_model('/content/skin_disease.h5')

# Function to load and preprocess the image
def load_and_preprocess_image(img_path, target_size=(128, 128)):
    img = image.load_img(img_path, target_size=target_size)
    img_array = image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0)
    img_array /= 255.0 # Assuming you normalized your images during training
    return img_array

# Function to predict the class of the image
def predict_image_class(model, img_array):
    prediction = model.predict(img_array)
    predicted_class = (prediction > 0.5).astype("int32")
    return predicted_class

# Function to print the prediction result
def print_prediction_result(predicted_class):
    class_labels = ['Normal Skin', 'Psoriasis Skin'] # Adjust this according to your class labels
    print(f"The uploaded image is classified as: {class_labels[predicted_class[0][0]]}")

# Use Colab's file upload widget
uploaded = files.upload()

for filename in uploaded.keys():
    img_path = filename
    img_array = load_and_preprocess_image(img_path, target_size=(128, 128)) # Adjust target_size
    predicted_class = predict_image_class(model, img_array)
    print_prediction_result(predicted_class)

# Display the uploaded image
img = image.load_img(img_path)
plt.imshow(img)
plt.axis('off')
plt.show()
```


 Choose Files img572.jpg

- **img572.jpg**(image/jpeg) - 75409 bytes, last modified: 6/13/2024 - 100% done

Saving img572.jpg to img572.jpg

1/1 [=====] - 0s 129ms/step

The uploaded image is classified as: Normal Skin



```
from google.colab import files
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing import image
from tensorflow.keras.models import load_model
import numpy as np

# Load your trained model
model = load_model('/content/skin_disease.h5')

# Function to load and preprocess the image
def load_and_preprocess_image(img_path, target_size=(128, 128)):
    img = image.load_img(img_path, target_size=target_size)
    img_array = image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0)
    img_array /= 255.0 # Assuming you normalized your images during training
    return img_array

# Function to predict the class of the image
def predict_image_class(model, img_array):
    prediction = model.predict(img_array)
    predicted_class = (prediction > 0.5).astype("int32")
    return predicted_class

# Function to print the prediction result
def print_prediction_result(predicted_class):
    class_labels = ['Normal Skin', 'Psoriasis Skin'] # Adjust this according to your class labels
    print(f"The uploaded image is classified as: {class_labels[predicted_class[0][0]]}")

# Use Colab's file upload widget
uploaded = files.upload()

for filename in uploaded.keys():
    img_path = filename
    img_array = load_and_preprocess_image(img_path, target_size=(128, 128)) # Adjust target_size
    predicted_class = predict_image_class(model, img_array)
    print_prediction_result(predicted_class)

# Display the uploaded image
img = image.load_img(img_path)
plt.imshow(img)
plt.axis('off')
plt.show()
```



Choose Files img27.jpg

- **img27.jpg**(image/jpeg) - 140003 bytes, last modified: 6/13/2024 - 100% done

Saving img27.jpg to img27.jpg

1/1 [=====] - 0s 146ms/step

The uploaded image is classified as: Normal Skin

